

## # FamilyHub backend profile update

### ## Resumen

- Endpoint implementado: PATCH /users/me
- Alias adicional: PATCH /api/users/me
- Auth: JWT obligatorio por Authorization: Bearer <token>
- Request: multipart/form-data
- Campos soportados:
- nombre?
- avatar?
- notification\_prefs? como JSON string

### ## Archivos modificados

- backend/index.js
- backend/src/controllers/users.controller.js
- backend/src/middleware/multiPartForm.js
- backend/src/routes/users.js
- backend/src/lib/sharp.js
- docs/familyhub-backend-profile-update-storage-policies.sql

### ## Endpoint implementado

- Valida JWT con el middleware ya existente.
- Parsea multipart/form-data en memoria.
- Rechaza payload no multipart.
- Rechaza avatar mayor a 5MB.
- Rechaza avatar que no sea imagen.
- Comprime avatar a WebP.
- Intenta calidad 80 y baja de forma escalonada hasta cumplir <= 500KB.
- Sube al bucket avatars con upsert a la key \${user\_id}/avatar.webp.
- Obtiene URL pública del avatar.
- Hace merge parcial de notification\_prefs.
- Actualiza user\_metadata en Supabase Auth.
- Devuelve el usuario actualizado completo en usuario.

### ## Decisiones técnicas

- No hay tabla profiles ni users modelada en el repo.
- signUp ya guardaba nombre en user\_metadata.
- Para no inventar una tabla, la actualización se resolvió sobre auth.users.user\_metadata.
- Se guardan:
- nombre
- notification\_prefs
- avatar\_url
- avatar\_path

### ## SQL de policies

Archivo:

- docs/familyhub-backend-profile-update-storage-policies.sql

Incluye:

- creación/ajuste del bucket avatars
- lectura para usuarios autenticados
- insert/update/delete solo dentro de la carpeta del propio usuario

## Ejemplo de request multipart

bash

```
curl --request PATCH "http://localhost:3000/users/me" \  
--header "Authorization: Bearer TU_JWT" \  
--form "nombre=Valeria" \  
--form "notification_prefs={\"feed\":true,\"gps\":false}" \  
--form "avatar=@C:/ruta/avatar.jpg"
```

## Test manual

1. Iniciar backend.
2. Hacer login y copiar el access token.
3. Ejecutar el PATCH /users/me con una foto real.
4. Confirmar 200 OK.
5. Revisar usuario.user\_metadata.avatar\_url.
6. Abrir esa URL en navegador.
7. Confirmar que carga la imagen.
8. Repetir con otra foto y confirmar que se sobrescribe la misma ruta.
9. Probar notification\_prefs parcial y verificar merge.
10. Probar JSON inválido y validar 400.

## Riesgos menores

- El entorno local no permitió instalar multer desde npm. El parseo multipart quedó resuelto en memoria con APIs nativas de Node para no bloquear el endpoint.
- La consigna pide URL pública y al mismo tiempo lectura solo autenticada. En Supabase, una URL pública implica bucket público. Si querés lectura estrictamente autenticada, hay que pasar a signed URLs en vez de getPublicUrl.